Técnicas de Programação II
Sub-algoritmos

Sub-algoritmos

- Em geral, um programa é executado linearmente, uma linha após a outra, até o fim
- Entretanto, quando são utilizados sub-algoritmos, é possível a realização de desvios na execução natural dos programas
- Assim, um programa é executado linearmente até a chamada de um sub-algoritmo
- O programa que chama um sub-algoritmo ("chamador") é temporariamente suspenso e o controle é passado para o sub-algoritmo, que é executado
- Ao terminar o sub-algoritmo, o controle retorna para o programa que realizou a chamada ("chamador")
- Tipos de Sub-algoritmos:
  - Funções (*functions*)
  - Procedimentos (*procedures*)

Funções

- É comum encontrar-se nas linguagens de programação, várias funções embutidas, por exemplo, **sin** (seno), **cos** (co-seno), **abs** (valor absoluto), **sqrt** (raiz quadrada)
- Funções embutidas podem ser utilizadas diretamente em expressões
- Por exemplo, o comando:
  - hipotenusa ← sqrt(cateto1^2 + cateto2^2)
  - calcula a hipotenusa de um triângulo retângulo como a raiz quadrada da soma dos quadrados dos dois catetos

Funções

- Essas funções são utilizadas em expressões como se fossem *simplesmente* variáveis comuns
- Como variáveis comuns, as funções têm (ou *retornam*) **um único valor**
- É responsabilidade do programador fornecer os *argumentos* (ou *parâmetros*) necessários para a função efetuar seus cálculos
- Por exemplo
  - A função **abs** tem como parâmetro um número, retornando um valor numérico
  - Em C/C++, a função **pow** tem como parâmetros dois números, retornando um valor numérico
  - Em C/C++, a função **strlen** tem como parâmetro uma string, retornando um valor numérico inteiro

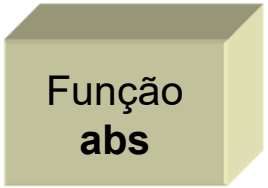
Funções

- A utilização de funções afeta o fluxo de controle em um programa
- Quando uma função é chamada, o programa que chama a função fica em estado de espera e o controle passa para as instruções que definem a função
- Após a execução da função com os parâmetros fornecidos, o controle retorna ao ponto de chamada da função, com o valor calculado pela função

Função: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Memória	
Endereço	Valor



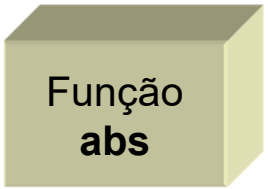
Instruções definindo a função **abs**

Instruções definindo a função **sqrt**

Função: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Memória	
Endereço	Valor
x	
y	
valor	



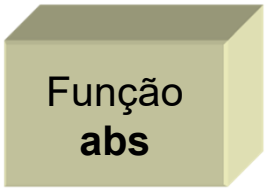
Instruções definindo a função **abs**

Instruções definindo a função **sqrt**

Função: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Memória	
Endereço	Valor
x	-10
y	
valor	



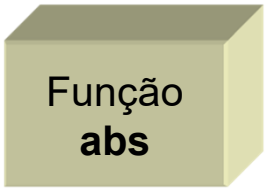
Instruções definindo a função **abs**

Instruções definindo a função **sqrt**

Função: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Memória	
Endereço	Valor
x	-10
y	16
valor	



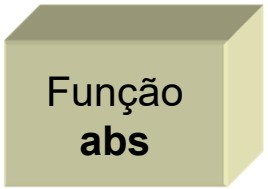
Instruções definindo a função **abs**

Instruções definindo a função **sqrt**

Função: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Memória	
Endereço	Valor
x	-10
y	16
valor	



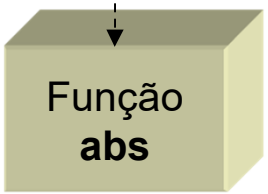
Instruções definindo a função **abs**

Instruções definindo a função **sqrt**

Função: Fluxo de Controle

O controle é transferido para a função **abs**

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```



Instruções definindo a função **abs**

Memória	
Endereço	Valor
x	-10
y	16
valor	

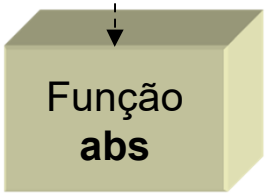
Instruções definindo a função **sqrt**

Função: Fluxo de Controle

O controle é transferido para a função **abs**

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Memória	
Endereço	Valor
x	-10
y	16
valor	



Instruções definindo a função **abs**

O código que define a função é executado, respeitando as estruturas de controle

Instruções definindo a função **sqrt**

Função: Fluxo de Controle

O controle é transferido para a função **abs**

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Retorno do controle

Retorno do valor calculado pela função (10)

Função **abs**

Instruções definindo a função **abs**

Memória	
Endereço	Valor
x	-10
y	16
valor	

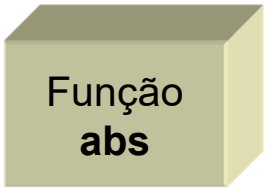
Função **sqrt**

Instruções definindo a função **sqrt**

Função: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Memória	
Endereço	Valor
x	-10
y	16
valor	



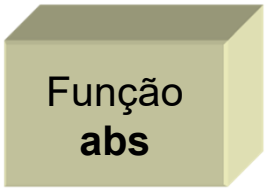
Instruções definindo a função **abs**

Instruções definindo a função **sqrt**

Função: Fluxo de Controle

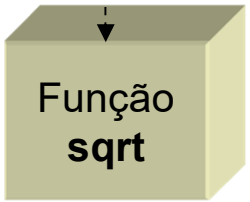
```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Memória	
Endereço	Valor
x	-10
y	16
valor	



Instruções definindo a função **abs**

O controle é transferido para a função **sqrt**

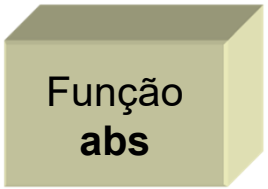


Instruções definindo a função **sqrt**

Função: Fluxo de Controle

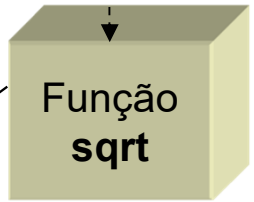
```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Memória	
Endereço	Valor
x	-10
y	16
valor	



Instruções definindo a função **abs**

O controle é transferido para a função **sqrt**



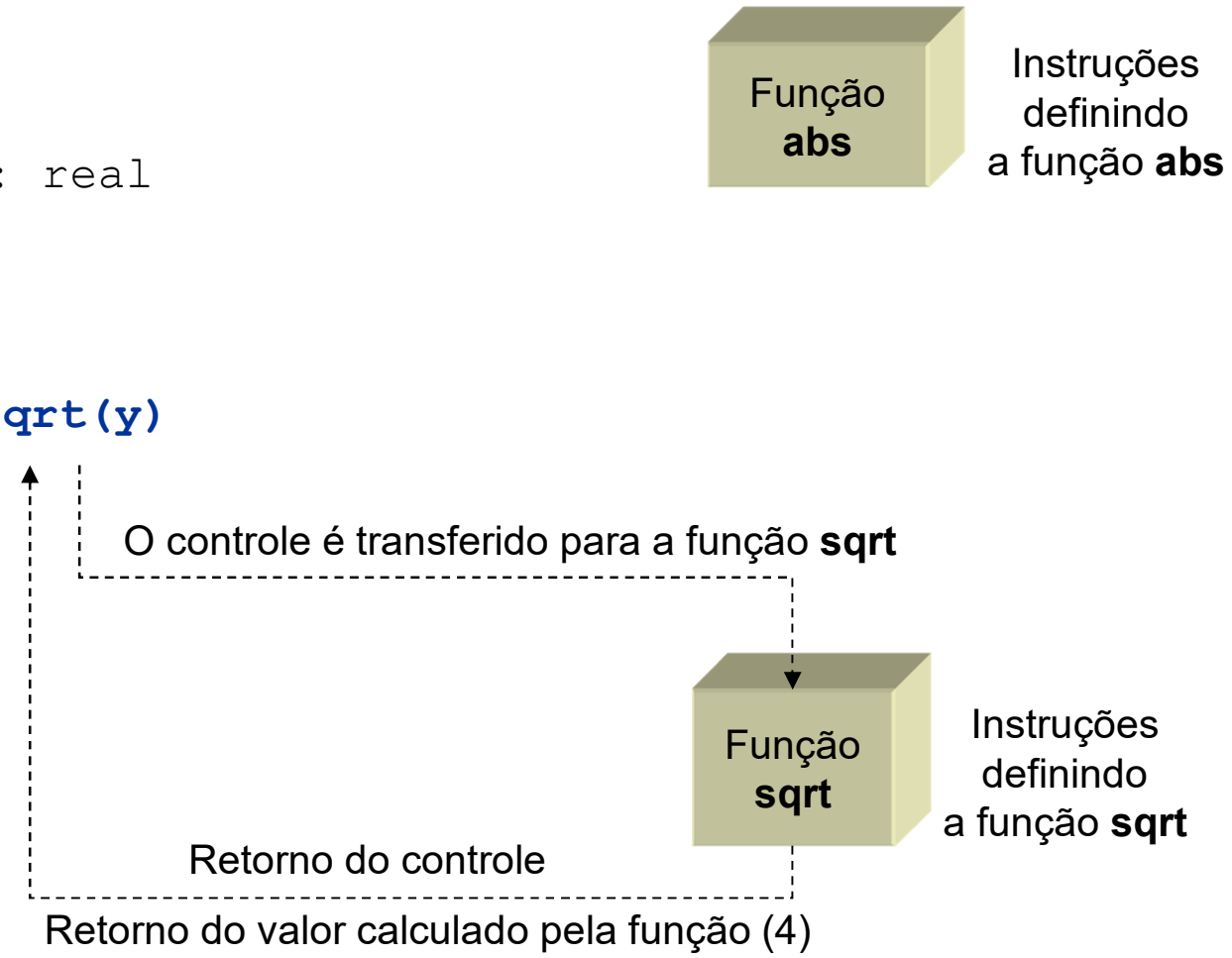
Instruções definindo a função **sqrt**

O código que define a função é executado, respeitando as estruturas de controle

Função: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

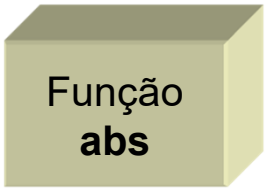
Memória	
Endereço	Valor
x	-10
y	16
valor	



Função: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Memória	
Endereço	Valor
x	-10
y	16
valor	14



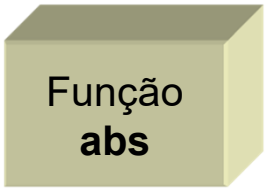
Instruções definindo a função **abs**

Instruções definindo a função **sqrt**

Função: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Memória	
Endereço	Valor
x	-10
y	16
valor	14



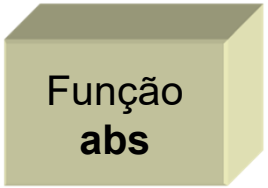
Instruções definindo a função **abs**

Instruções definindo a função **sqrt**

Função: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y,valor : real

  x ← -10
  y ← 16
  valor ← abs(x) + sqrt(y)
  Escreva(valor)
Fim
```

Memória	
Endereço	Valor



Instruções definindo a função **abs**

Instruções definindo a função **sqrt**

Funções

- Em algumas situações, o programador gostaria de utilizar (definir) novas funções
- Por analogia, na Matemática, escreve-se (ou *define-se*) uma função por meio de parâmetros, por exemplo:
  - $f(x) = x^2 - 3x + 2$ *Definição da função f*
  - Esta função *f* foi definida em termos do parâmetro *x*
- Para saber o valor da função para um valor particular do argumento *x*, por exemplo, *x* = 3, basta substituir este valor onde aparece o parâmetro *x*:
  - $f(3) = 3^2 - 3(3) + 2 = 2$ *"Ativação" da função*
  - $f(1) = 1^2 - 3(1) + 2 = 0$
  - $f(-1) = (-1)^2 - 3(-1) + 2 = 6$
- Uma vez definida a nova função, ela pode ser utilizada sempre que necessária, mesmo em outras (novas) funções

Funções

- Como na Matemática, os parâmetros podem ser nomeados livremente
  - Por exemplo, são equivalentes as funções
    - $f(x) = x^2 – 3x + 2$
    - $f(y) = y^2 – 3y + 2$
- O nome da função é definido pelo programador e segue a mesma norma de formação de identificadores
- Por exemplo, são equivalentes as funções
  - $f(x) = x^2 – 3x + 2$
  - $g(x) = x^2 – 3x + 2$
  - $f(y) = y^2 – 3y + 2$
  - $g(y) = y^2 – 3y + 2$

Funções

- Funções podem ter mais de um parâmetro (argumento):
  - $g(x,y) = x^2 + y^3$
    - **g** possui 2 parâmetros
  - $h(x,y,z) = x^2 + 2y + z^2$
    - **h** possui 3 parâmetros
- Pode-se avaliar cada uma dessas funções de forma análoga:
  - $g(3,2) = 3^2 + 2^3 = 9 + 8 = 17$
  - $h(1,3,2) = 1^2 + 2(3) + 2^2 = 1 + 6 + 4 = 11$
- Notar a correspondência estabelecida entre os parâmetros da definição da função e os parâmetros de ativação (ou execução) da função
- No caso da função *g*, 3 é substituído para cada ocorrência de *x* e 2 é substituído para cada ocorrência de *y.* Essa ordem é fundamental, pois *g(3,2)* **não** é o mesmo que *g(2,3)*

Exemplo

```
//-------------------------------------------------------
// Encontra o máximo entre dois inteiros
Função Maximo(x,y : inteiro) : inteiro
  declare max : inteiro

  Se (x > y) Então
    max ← x
  Senão
    max ← y
  Fim Se
  Retorne(max)
Fim Função
//-------------------------------------------------------
Algoritmo TesteMaximo
Início
  declare a,b : inteiro

  a ← 5
  b ← 10
  Escreva(”Máximo de ”,a,” e ”,b,”=”,Maximo(a,b))
  Escreva(”Máximo de ”,a+10,” e ”,b-5,”=”,Maximo(a+10,b-5))
Fim
```

Outro Exemplo

```
//-------------------------------------------------------
// Encontra o máximo entre dois inteiros
Função Maximo(x,y : inteiro) : inteiro
  declare max : inteiro

  Se (x > y) Então
    max ← x
  Senão
    max ← y
  Fim Se
  Retorne(max)
Fim Função
//-------------------------------------------------------
// Encontra o máximo entre tres inteiros
Função Maximo3(a,b,c : inteiro) : inteiro
  Retorne(Maximo(a,Maximo(b,c)))
Fim Função
//-------------------------------------------------------
Algoritmo TesteMaximo
Início
  declare a,b,c : inteiro

  a ← 5
  b ← 10
  c ← 15
  Escreva(Maximo3(a,b,c))
Fim
```

Sub-algoritmos em C++

- A declaração (definição) de um sub-algoritmo (função ou procedimento) em C++ é similar à do programa principal (**main**)
- Sub-algoritmos preferencialmente devem ser declarados antes do programa que os chama
- É permitido declarar variáveis dentro do sub-algoritmo
  - As variáveis declaradas dentro do sub-algoritmo, incluindo os parâmetros são denominadas **variáveis locais**

Funções em C++

- Função $f(x) = x^2 - 3x + 2$

```
float f(float x)
{
  return(x * x - 3 * x + 2);
}
```

Funções em C++

- Função $f(x) = x^2 - 3x + 2$

```
float f(float x)
{
   return(x * x - 3 * x + 2);
}
```

Indica o tipo de dado que a função deve retornar. Neste caso, a função retorna um número real

Funções em C++

- Função $f(x) = x^2 - 3x + 2$

```
float f(float x)
{
   return(x * x - 3 * x + 2);
}
```

Indica o nome da função. Neste caso, f

Funções em C++

- Função $f(x) = x^2 - 3x + 2$

```
float f(float x)
{
   return(x * x - 3 * x + 2);
}
```

Indica o tipo de dado do primeiro parâmetro da função. Neste caso é um número real

Funções em C++

- Função $f(x) = x^2 - 3x + 2$

```
float f(float x)
{
  return(x * x - 3 * x + 2);
}
```

Indica o nome do primeiro parâmetro da função. Neste caso, x

Funções em C++

- Função $f(x) = x^2 - 3x + 2$

```
float f(float x)
{
   return(x * x - 3 * x + 2);
}
```

Indica o que deve ser retornado pela função

Funções em C++

- Função $f(x) = x^2 - 3x + 2$

```
float f(float x)
{ float r;

  r = x * x - 3 * x + 2;
  return r;
}
```

Se houver necessidade, variáveis (locais) adicionais podem ser declaradas dentro da função, de forma análoga à declaração de variáveis no programa principal

Exemplo de Chamada em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------------
float f(float x)                /* definicao da funcao f */
{
  return x * x - 3 * x + 2;
}
//-------------------------------------------------------------
int main()                      /* programa principal */
{ float a,r;

  a = 3;
  /* utilizar a funcao numa expressao simples */
  r = f(a);                              /* ativacao de f */
  cout << ”Valor de f(“ << a << “)=” << r << endl;
  /* utilizar a funcao numa expressao mais elaborada */
  r = f(2*a) + 3 * f(1) + 2 * f(-1); /* ativacao de f */
  cout << r << endl;
  return 0;
}
```

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor

Memória	
Endereço	Valor

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor

Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor

Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	3
divisor	4

Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	3
divisor	4
quociente	

Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	3
divisor	4
quociente	0.75

Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	3
divisor	4
quociente	0.75

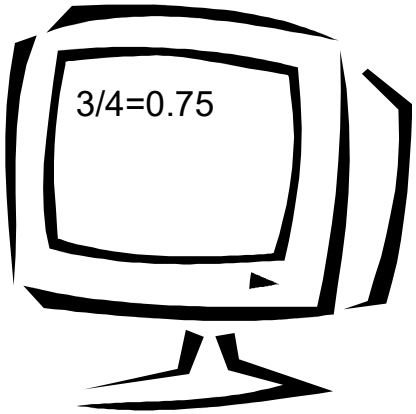
Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor

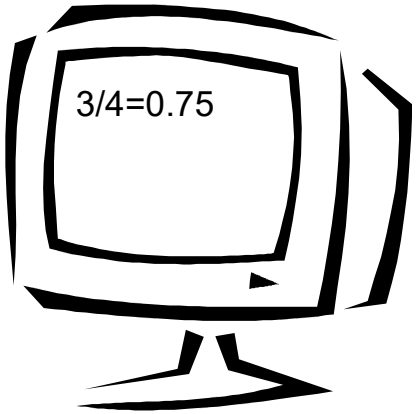
Memória	
Endereço	Valor
a	3
b	4



Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor

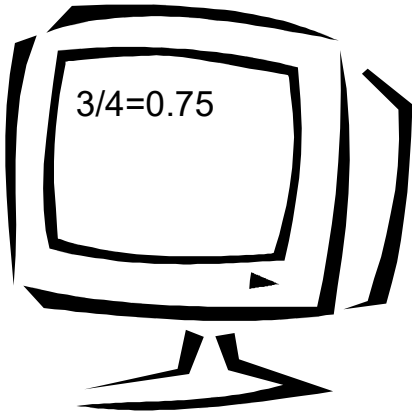


Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	4
divisor	3

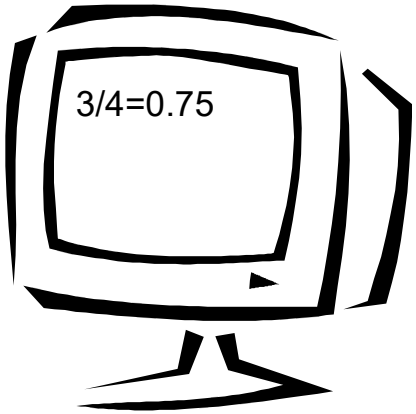


Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	4
divisor	3
quociente	

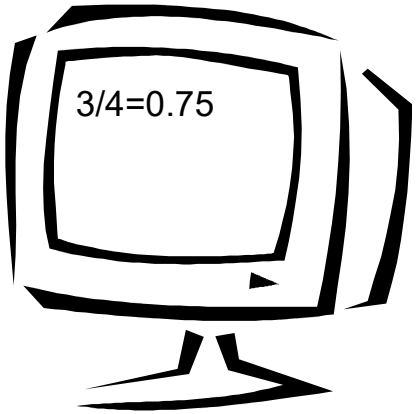


Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	4
divisor	3
quociente	1.33

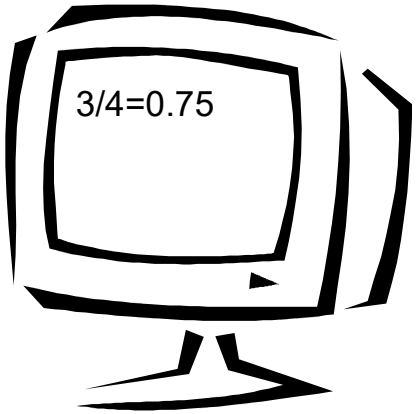


Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	4
divisor	3
quociente	1.33

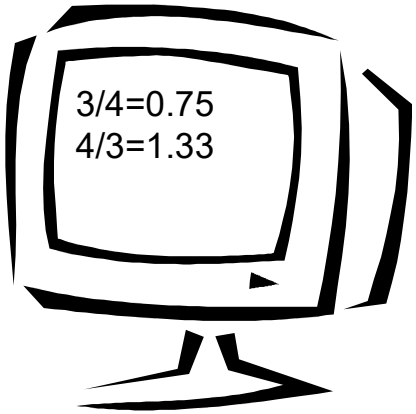


Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor

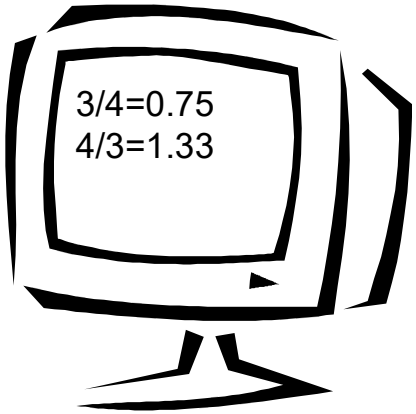


Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor



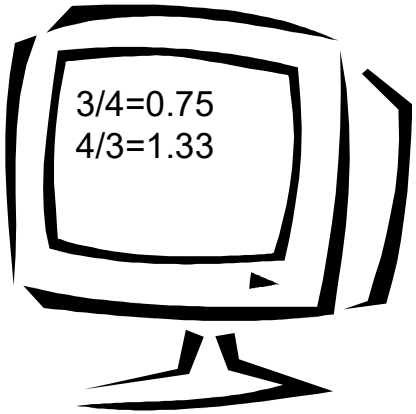
Memória	
Endereço	Valor
a	3
b	4

Funções em C++

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
float divisao(int dividendo, int divisor)
{ float quociente;

  quociente = 1.0 * dividendo / divisor;
  return quociente;
}
//-------------------------------------------------------
int main()
{ int a=3, b=4;

  cout << a << "/" << b << "="
       << divisao(a,b) << endl;
  cout << b << "/" << a << "="
       << divisao(b,a) << endl;
  return 0;
}
```

Memória	
Endereço	Valor

Memória	
Endereço	Valor



Exemplo

- A constante especial π desempenha um importante papel na matemática. Não é surpresa que haja muitos métodos de obter aproximações numéricas de π. Muitas destas aproximações envolvem operações com séries infinitas. Dentre essa séries temos:

$$\pi = 4\sum_{i=0}^{\infty} \frac{(-1)^i}{2i+1}$$

- Para cálculos práticos, as séries infinitas devem terminar após um número finito de termos, penalizando a precisão do resultado
- Preparar uma função para calcular π de acordo com a série acima. A função deve aceitar, como parâmetro, o valor N, indicando o número de termos a serem utilizados nos cálculos

Exemplo

$$\pi = 4\sum_{i=0}^{\infty}\frac{(-1)^i}{2i+1}$$

```
//-------------------------------------------------------
// Calcula o valor de Pi usando serie com N termos
Função Pi(N : inteiro) : real
  declare soma       : real    // variáveis locais
          i,sinal    : inteiro

  soma ← 0.0
  sinal ← 1
  Para i ← 0 até N-1 Faça
    soma ← soma + sinal/(2.0*i+1.0)
    sinal ← -sinal
  Fim Para
  retorne 4*soma
Fim Função
//-------------------------------------------------------
Algoritmo ImprimePi. Testa função Pi.
Início
  declare A : inteiro

  Escreva(”Numero de termos = ”)
  Leia(A)
  Escreva(”Valor de Pi = ”,Pi(A),” com ”,A,” termos”)
Fim
```

Exemplo em C++

$$\pi = 4\sum_{i=0}^{\infty}\frac{(-1)^i}{2i+1}$$

```
#include <iostream>
using namespace std;
//-------------------------------------------------------
// Calcula o valor de Pi usando serie com N termos
float Pi(int N)
{ float soma=0.0;
  int i,sinal=1;

  for(i=0; i<=N-1; i++)
  { soma = soma + sinal/(2.0*i+1.0);
    sinal = -sinal;
  }
  return 4*soma;
}
//-------------------------------------------------------
// Testa função Pi.
int main()
{ int A;

  cout << ”Numero de termos = ”;
  cin >> A;
  cout << ”Valor de Pi = ” << Pi(A)
       << ” com ” << A << ” termos” << endl;
  return 0;
}
```